

DOCUMENT 4

Design, Implementation & Testing of Speed Optimized FFT algorithm on Softcore MicroBlaze Processor and Hardcore Zynq Processor

KAPISH GUPTA

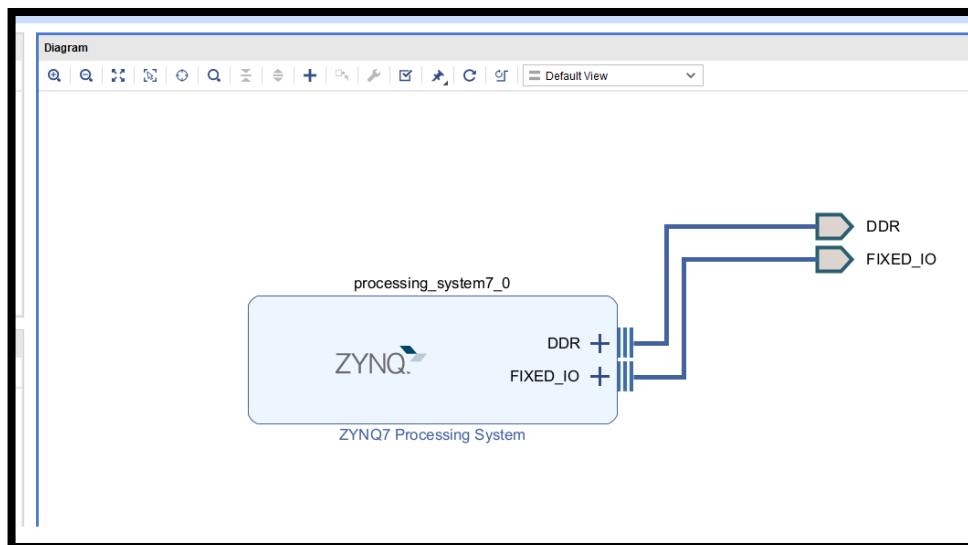
Objective

The goal is to measure exactly how long it takes to calculate the Fast Fourier Transform (FFT) purely in software using C code. We tested two different processors: the **hardcore Zynq ARM processor** and the **softcore MicroBlaze processor**.

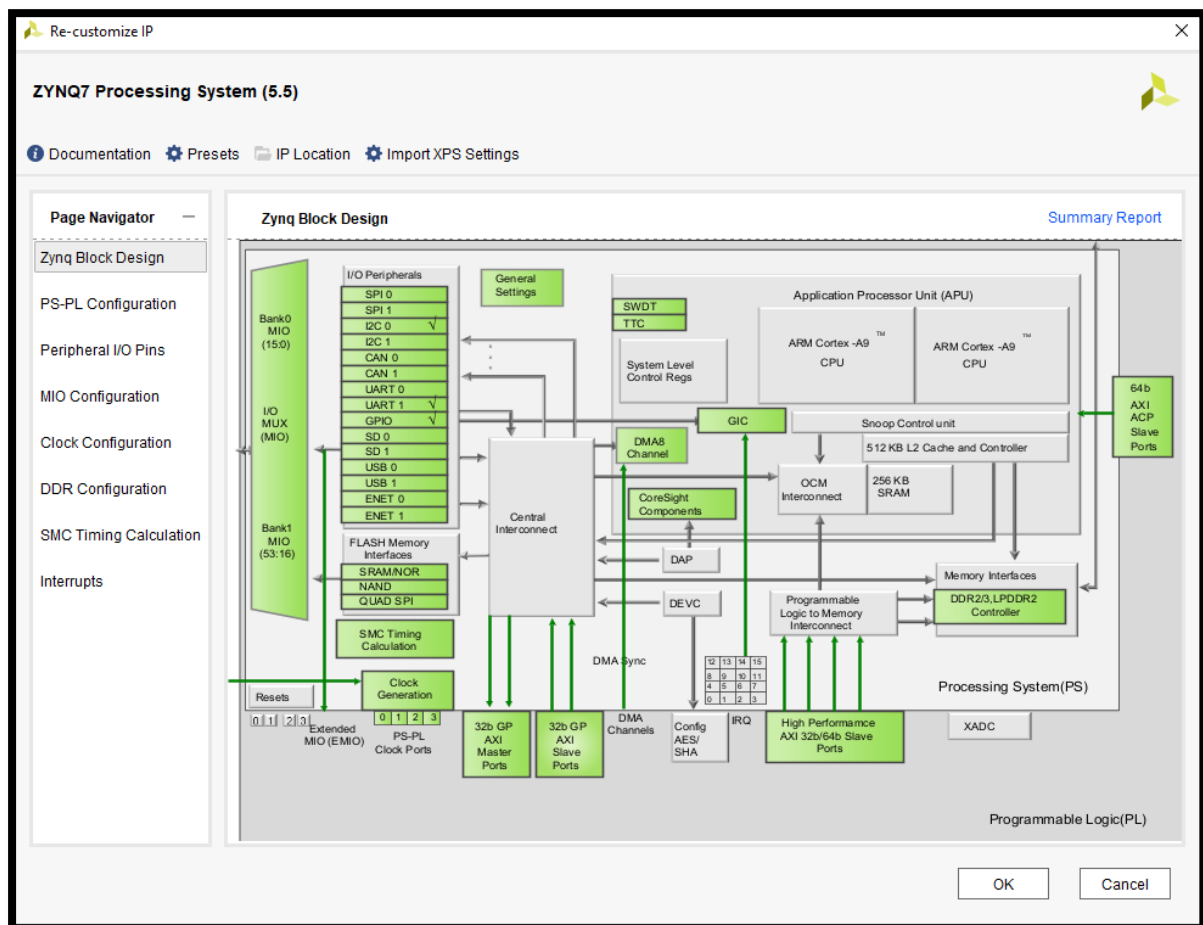
Zynq Hardcore Processor Configurations

Because the Zynq ARM Cortex-A9 is a physical chip, we do not need to add hardware units. The only configuration required in Vivado is adjusting the clock speed of the Processing System (PS).

Block diagram

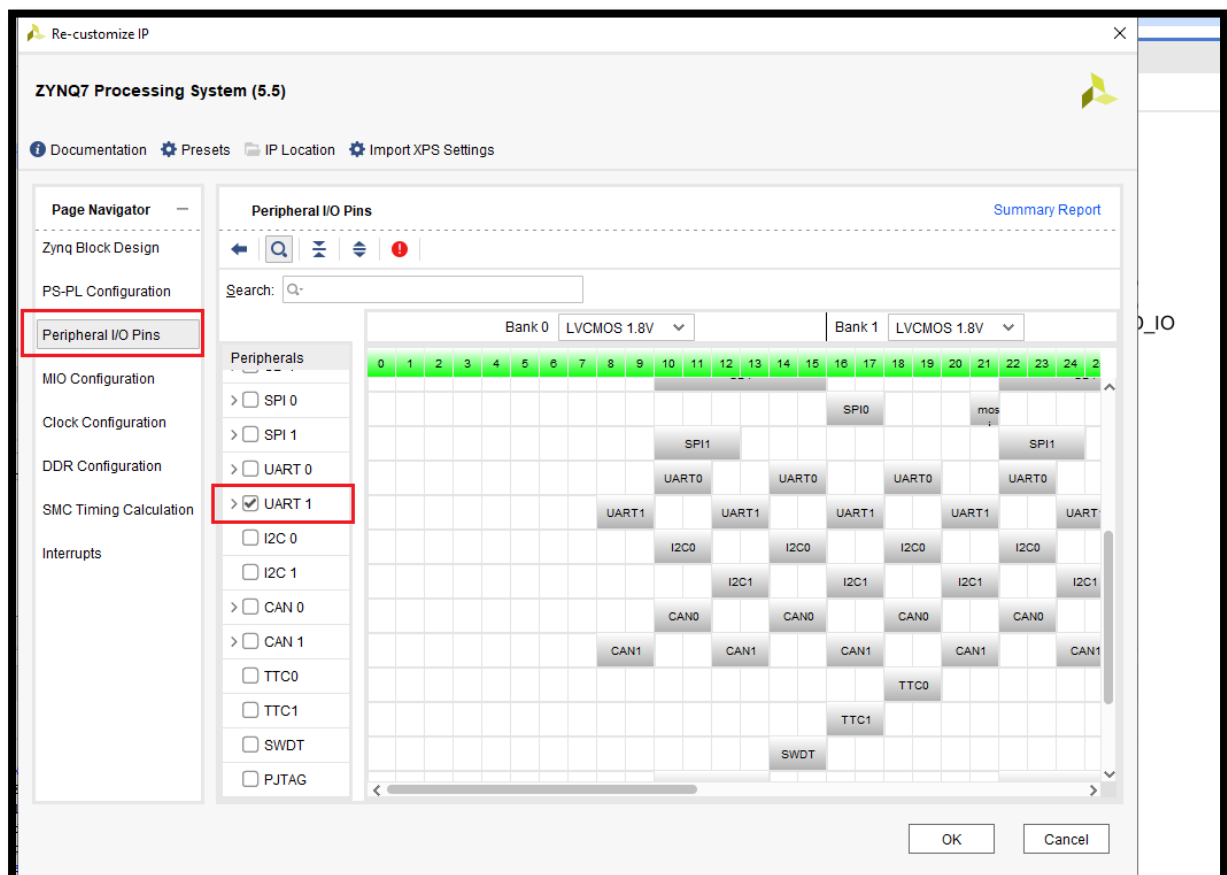


Double click on the IP to customize it!



Selecting the Components

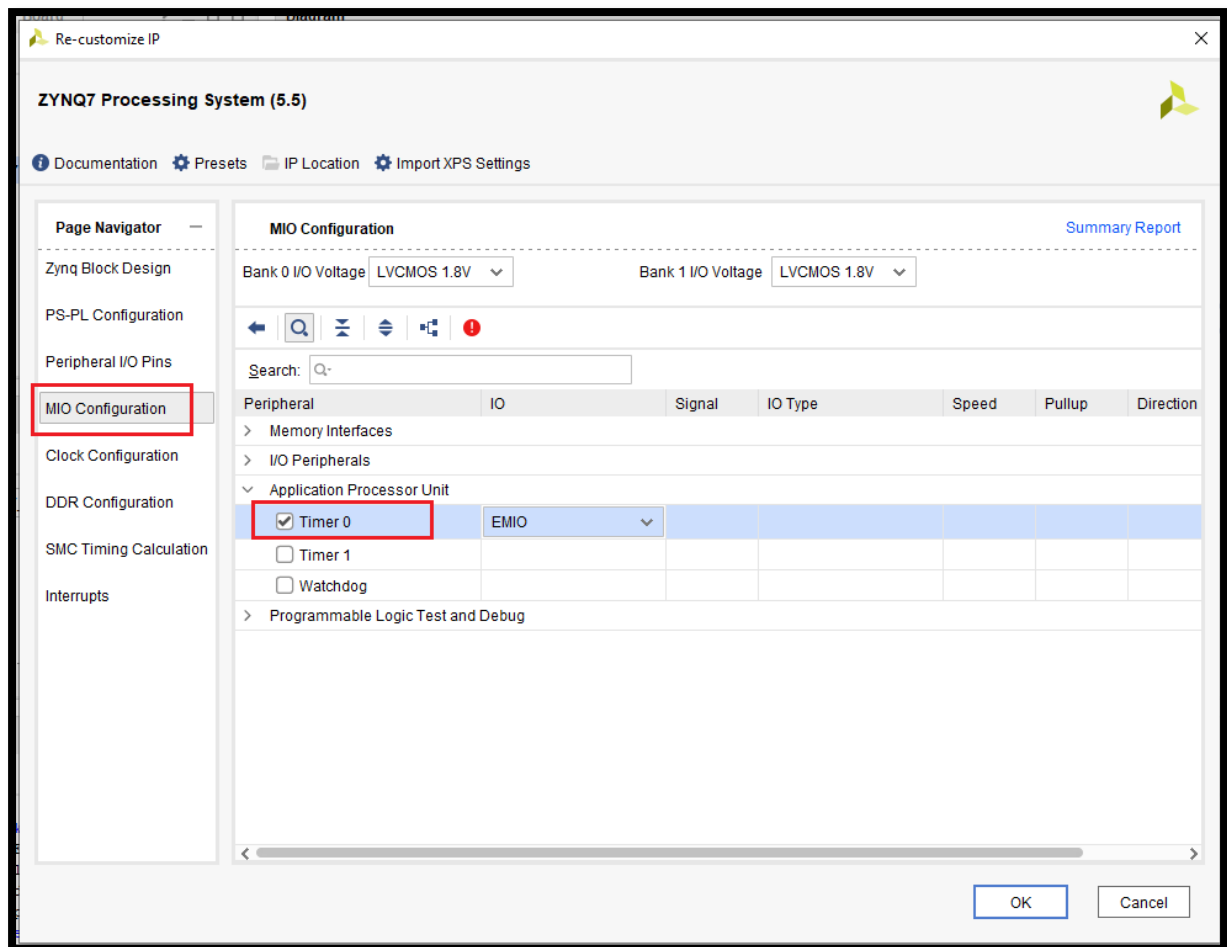
Inside the configuration menu, we went to the Peripheral I/O Pins section. This screen allows us to turn different parts of the chip ON or OFF. For this project, we made sure the essential components—like the UART (so we can print text to our screen)—were enabled.



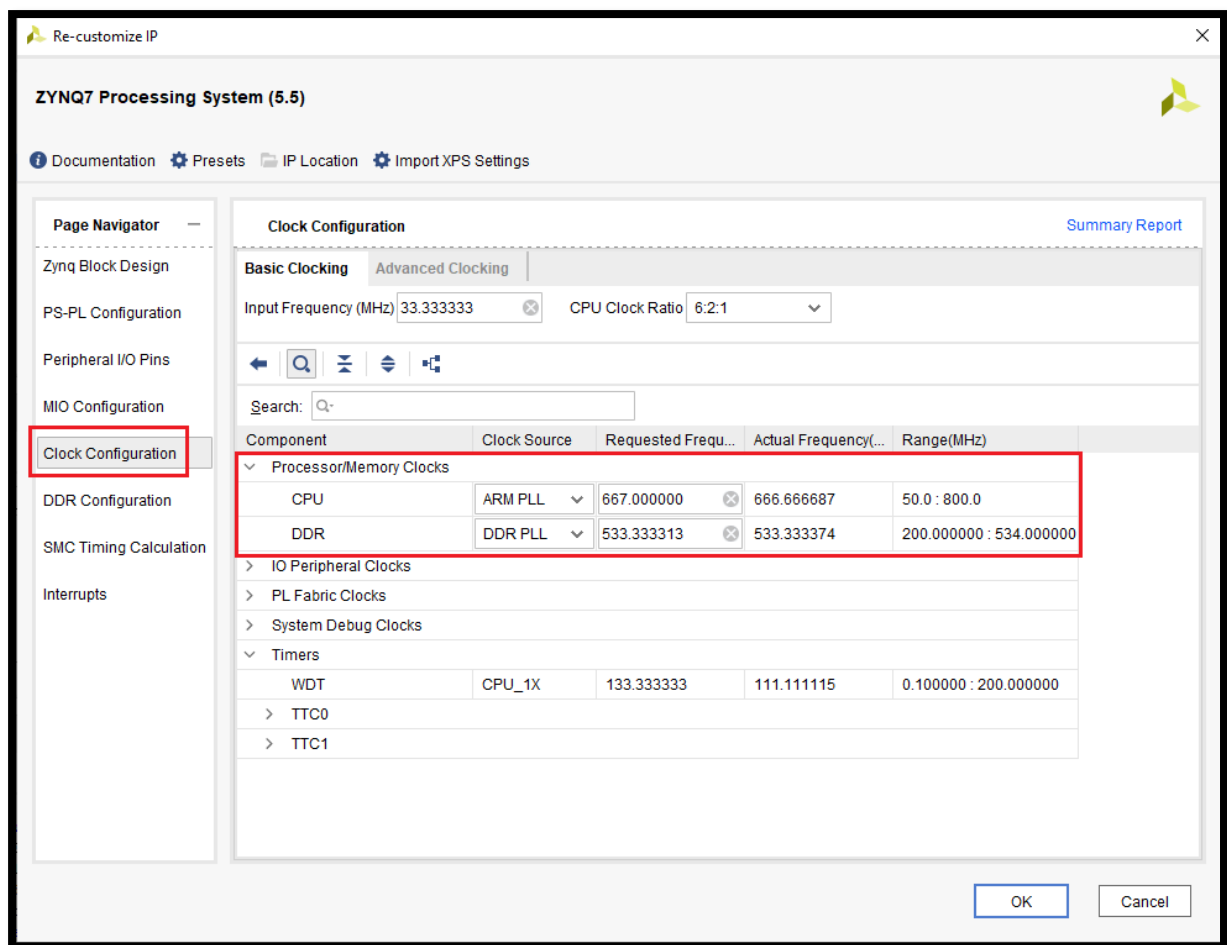
Enable the **Timer-0** to calculate the execution time of the FFT algorithm.

EMIO stands for Extended Multiplexed IO.

As the **Arm-Cortex A9** is a **32-bit processor**, Timer is also of 32-bits.

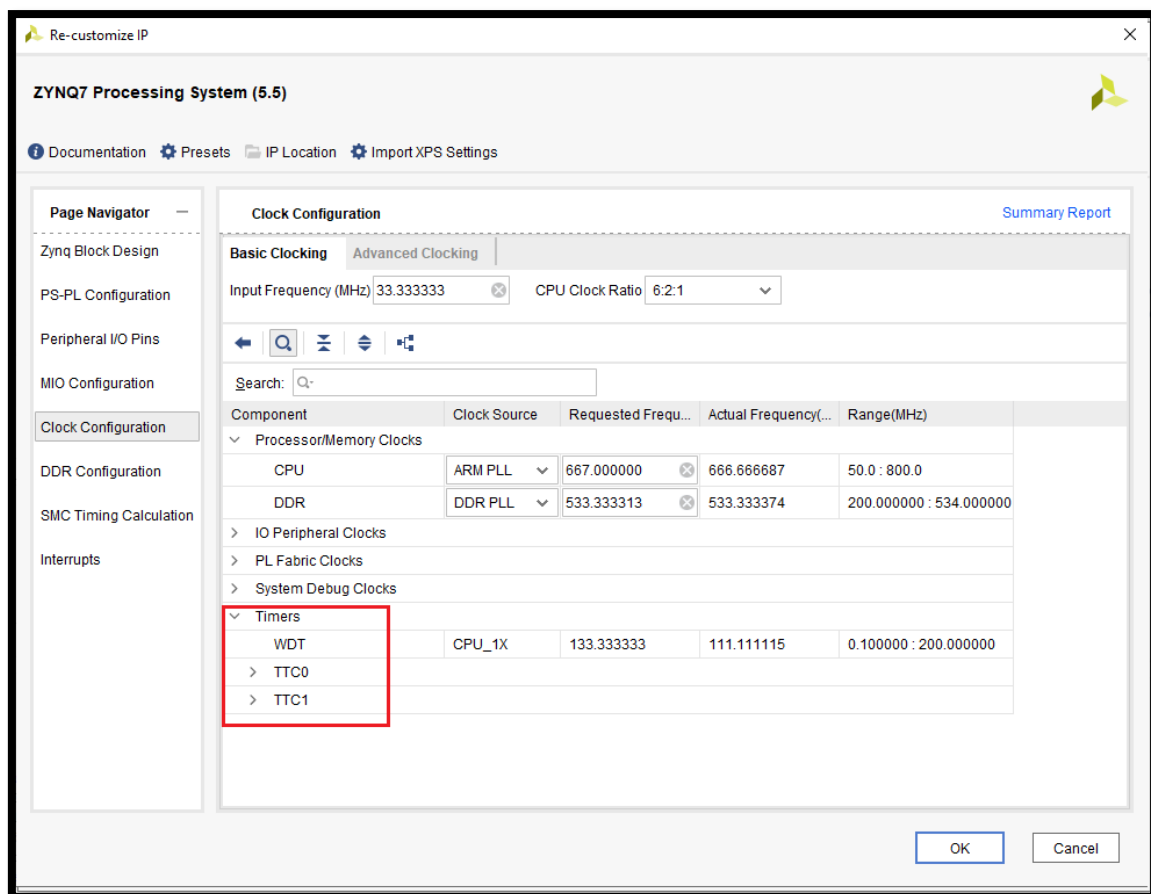


Now, go to clock configuration menu and set the frequency of the clock. By default, it is set to 667 MHz.



Here, we can see that there is also timer option available in the same clock configuration menu.

There is a big difference between these clock configuration timers and MIO configuration's APU timer.



1. The APU Timers (Private Timers & Watchdogs)

These are **private timers** built directly into the brain of the ARM processor.

- Location: These timers sit right next to the Cortex-A9 cores.
- Function: Every ARM core gets its own private timer and its own private watchdog. They are mainly used by the processor itself (or an operating system like Linux) to keep track of its own internal schedule.
- Watchdog: The "Watchdog" is a strict safety feature. It counts down continuously. The software running on the processor must constantly reset this timer. If your C code freezes or crashes and fails to reset it, the watchdog timer hits zero and automatically reboots the processor to fix the freeze.

2. The TTC0 and TTC1 (Triple Timer Counters)

These are **shared, general-purpose** 16-bit timers.

- Location: They are located in the main peripheral section of the Zynq chip, not locked privately inside the CPU cores.
- Function: "Triple" means each block (TTC0 and TTC1) actually contains 3 separate, independent timers. These are used for interacting with the outside world or timing general hardware events. For example, you would use a TTC to generate a PWM (Pulse Width Modulation) signal to control a motor, or to measure exactly how long an external physical button was pressed.
- Routing: TTCs can be wired directly to the external pins of your board or routed directly into the FPGA programmable logic.

Feature	APU Timers (Watchdog / Timer 0 & 1)	TTC0 / TTC1 (Triple Timer Counters)
Ownership	Private (Belongs strictly to a specific CPU core).	Shared (Can be used by the whole system).
Main Purpose	Internal CPU scheduling, OS ticks, and system crash recovery.	External hardware events, generating signals (PWM), and general timing.
Physical Routing	Cannot connect to the outside world.	Can be wired to external board pins or the FPGA logic.

C Code of the FFT algorithm

1. Libraries included

```
183 #include <stdio.h>
184 #include <stdlib.h>
185 #include <complex.h>
186 #include <math.h>
187 #include <xtime_l.h>
188 #define FFT_Size 65536
189
190 #define input_sel 1
191 #define fs      100000000.0 // 100 MHz
192 #define f       1000000.0   // 1 MHz
193
194 #ifndef M_PI
195 #define M_PI 3.14159265358979323846
196 #endif
197
```

2. generate_input(): The Signal generator

The computer needs data to perform math on. This function creates a signal (like a sine wave/ sinc wave/ square wave) and fills an array called FFT_input. It calculates exactly where each point of the wave belongs based on the sampling frequency (fs) and the signal frequency (f).

```
199 void generate_input(float complex* input, int sel, int N) {
200     switch (sel) {
201         case 1: {
202             //sine wave
203             int samples_per_cycle = (int)ceil(fs / f);
204             int total_cycles = (int)ceil(N / (float)samples_per_cycle);
205             int index = 0;
206
207             for (int cycle = 0; cycle < total_cycles; cycle++) {
208                 for (int sample = 0; sample < samples_per_cycle; sample++) {
209                     if (index < N) {
210                         float time = (float)index / fs;
211                         float x = 2.0 * (float)M_PI * f * time;
212                         input[index] = cosf(x) + sinf(x) * I;
213                         index++;
214                     }
215                 }
216             }
217             break;
218         }
219         case 2: {
220             //rect wave
221             int samples_per_cycle = (int)ceil(fs / f);
222             int total_cycles = (int)ceil(N / (float)samples_per_cycle);
223             int index = 0;
224
225             for (int cycle = 0; cycle < total_cycles; cycle++) {
226                 for (int sample = 0; sample < samples_per_cycle; sample++) {
227                     if (index < N) {
228                         // High for the first half of the cycle, Low for the second half
229                         if (sample < (samples_per_cycle / 2)) {
230                             input[index] = 1.0 + 0.0 * I;
231                         } else {
232                             input[index] = 0.0 + 0.0 * I;
233                         }
234                         index++;
235                     }
236                 }
237             }
238             break;
239         }
240     }
241 }
242
```

```

243
244     case 3: {
245         int samples_per_cycle = (int)ceil(fs / f);
246         int total_cycles = (int)ceil(N / (float)samples_per_cycle);
247         int index = 0;
248
249         for (int cycle = 0; cycle < total_cycles; cycle++) {
250             int cycle_center = samples_per_cycle / 2;
251             float stretch_factor = (float)samples_per_cycle / 8.0;
252             for (int sample = 0; sample < samples_per_cycle; sample++) {
253                 if (index < N) {
254                     float val = 0.0;
255                     if (sample == cycle_center) {
256                         val = 1.0;
257                     } else {
258                         float x = (float)M_PI * (float)(sample - cycle_center) / stretch_factor;
259                         val = sinf(x) / x;
260                     }
261                     input[index] = val + 0.0 * I;
262                     index++;
263                 }
264             }
265             break;
266         }
267     default: {
268         for (int i = 0; i < N; i++) {
269             input[i] = 0.0f + 0.0f * I;
270         }
271         break;
272     }
273 }
274 }
275 }
276

```

3. reverse_index(): The Data Shuffle

The FFT algorithm requires the input data to be in a specific, jumbled order (called "bit-reversal"). This function takes the standard index of our array and mathematically flips the bits of the index number to rearrange the data so the FFT can process it and give the output in natural order.

```

336
337 // for N point fft, 2^k = N, so number of stages are k
338 unsigned int reverse_index(unsigned int value, int k){
339     unsigned int reverse_num = 0;
340     for(int i = 0; i < k; i++){
341         if(value & (1 << i)) {
342             reverse_num |= 1 << (k - 1 - i);
343         }
344     }
345     return reverse_num;
346 }
347

```

4. fft_N():

This is the heart of the project. It uses three nested for loops to process the data:

- **The Stages:** It divides the FFT into smaller and smaller groups.
- **The Butterflies:** This is the core mathematical operation. It combines two complex numbers (using a "twiddle factor") to calculate the frequency components of the signal.

```
347
348 // Pass N into the function
349 void fft_N(float complex* data, int N, float complex* input){
350
351
352     if(N <= 0 || (N & (N - 1))) return; // return the function if N=0,
353                                         // or not a power of 2.
354     int k = (int) log2(N);
355
356     for(int i = 0; i < N; i++){ //rearrange the input array in order
357                               //to get natural order output.
358         int reversed = reverse_index(i, k);
359         data[reversed] = input[i] ;
360     }
361
362     // (Stages) i acts as the step size.
363     for(int i = 2; i <= N; i *= 2){
364         int steps = i / 2;
365         // Blocks
366         for(int j = 0; j < N; j += i){
367             // Butterflies
368             for (int k = 0; k < steps; k++){
369                 // Twiddle factor calculation
370                 float angle = -2.0 * M_PI * k / i;
371                 float complex twiddleFact = cos(angle) + sin(angle) * I;
372
373                 float complex top_input = data[j + k];
374                 float complex bottom_input = data[j + k + steps];
375                 float complex T = bottom_input * twiddleFact;
376                 data[j + k] = top_input + T;
377                 data[j + k + steps] = top_input - T;
378             }
379         }
380     }
381 }
382
```

5. main():

This is where everything comes together. It creates the memory, calls the signal generator, starts the Timer, runs the `fft_N` function, stops the timer, and calculates the final execution time.

```
383 int main(){
384     int N=FFT_Size;
385     float complex* FFT_input = (float complex*) malloc (N* sizeof(float complex));
386     float complex* FFT_output= (float complex*) malloc (N* sizeof(float complex));
387
388     XTime start_time , end_time ; //declare the timer variables
389
390     float complex* ptr1 = FFT_input ;
391     float complex* ptr2 = FFT_output ;
392     generate_input(FFT_input, input_sel, N ) ;
393
394     XTime_SetTime(0) ; // Set the timer to 0
395
396     XTime_GetTime(&start_time) ; // get the value of timer at start
397
398     fft_N(FFT_output , N , FFT_input) ; // call the FFT function
399
400     XTime_GetTime(&end_time) ; // get the value of timer at end
401
402     float complex ip[FFT_Size] ; //declare 2 array for ip and output storage
403     float complex op[FFT_Size] ;
404
405     for (int i = 0 ; i < N ; i++){
406         ip[i] = *ptr1 ; //storing the input in ip array
407         ptr1++;
408     }
409
410     for (int i = 0 ; i < N ; i++){
411         float real2 = crealf(*ptr2)*crealf(*ptr2) ;
412         float imag2 = cimagf(*ptr2)*cimagf(*ptr2) ;
413         float magnitude = sqrt(real2+imag2) ; // calculate the magnitude of the output
414         op[i] = magnitude ; // and store in the op array
415         ptr2 ++ ;
416     }
417
418     float execution_time_sec = 0.0 ;
419     float execution_time_us = 0.0 ;
420     execution_time_sec = (float)1.0 * ( end_time - start_time ) / COUNTS_PER_SECOND ;
421     execution_time_us = execution_time_sec *(1000000) ;
422
423     return 0 ;
424 }
425
```

We use the **ARM Internal Timer** via the `xtime_l.h` library.

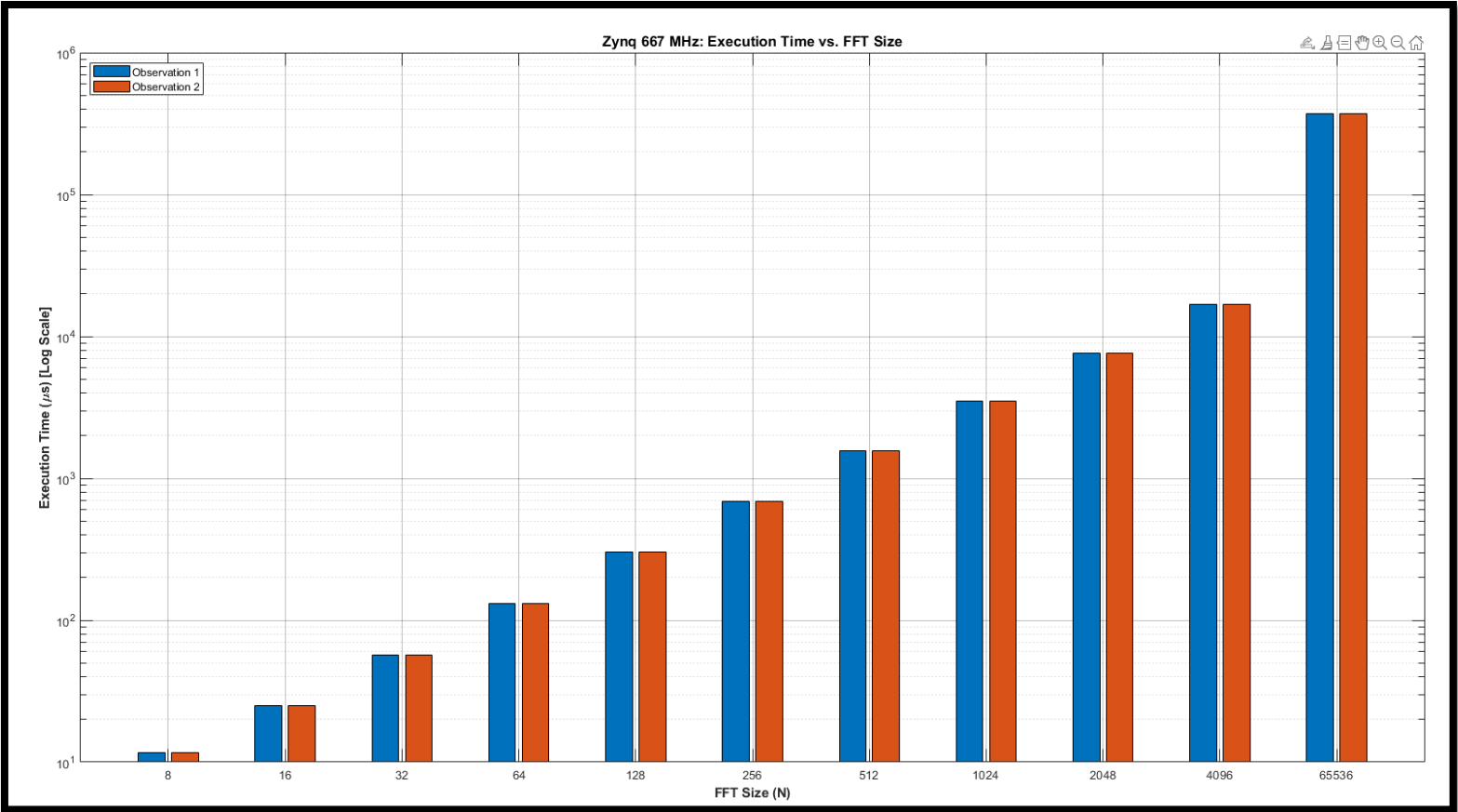
This timer is a built-in **64-bit hardware counter** that is always running in the background of the ARM processor.

1. **Setting the Timer:** `XTime_SetTime(0)` forces the global timer to start its count at zero.
2. **Capturing Start Time:** `XTime_GetTime(&start_time)` records the exact CPU clock count at the precise moment before the FFT begins.
3. **Capturing End Time:** `XTime_GetTime(&end_time)` records the clock count the moment the FFT finishes.
4. **Calculating Duration:** We calculate the difference between the end and start values (`end_time - start_time`).
5. **Converting to Seconds:** We divide this difference by `COUNTS_PER_SECOND` (a constant defined by the Zynq system). This gives us the execution time in actual seconds. Finally, we multiply that result by 1,000,000 to get the time in **microseconds (us)**.

Configuration A: Zynq at 667 MHz (Default)

Zynq: - Clock frequency 667 MHz

FFT Size	Time (in us) (in micro seconds)	
	Observation 1	Observation 2
8	11.58900	11.58890
16	25.14900	25.14920
32	56.96100	56.96300
64	131.63100	131.63000
128	303.46500	303.46900
256	692.83200	692.83270
512	1 565.82900	1 565.82500
1024	3 493.72800	3 493.73000
2048	7 678.56500	7 679.59900
4096	16 829.26000	16 830.98000
65536	369 704.40000	369 435.50000



Configuration B: Zynq at 100 MHz (Down clocked)

Re-customize IP

ZYNQ7 Processing System (5.5)

Documentation Presets IP Location Import XPS Settings

Page Navigator

- Zynq Block Design
- PS-PL Configuration
- Peripheral I/O Pins
- MIO Configuration
- Clock Configuration**
- DDR Configuration
- SMC Timing Calculation
- Interrupts

Clock Configuration [Summary Report](#)

Basic Clocking Advanced Clocking

Input Frequency (MHz) 33.333333 CPU Clock Ratio 6:2:1

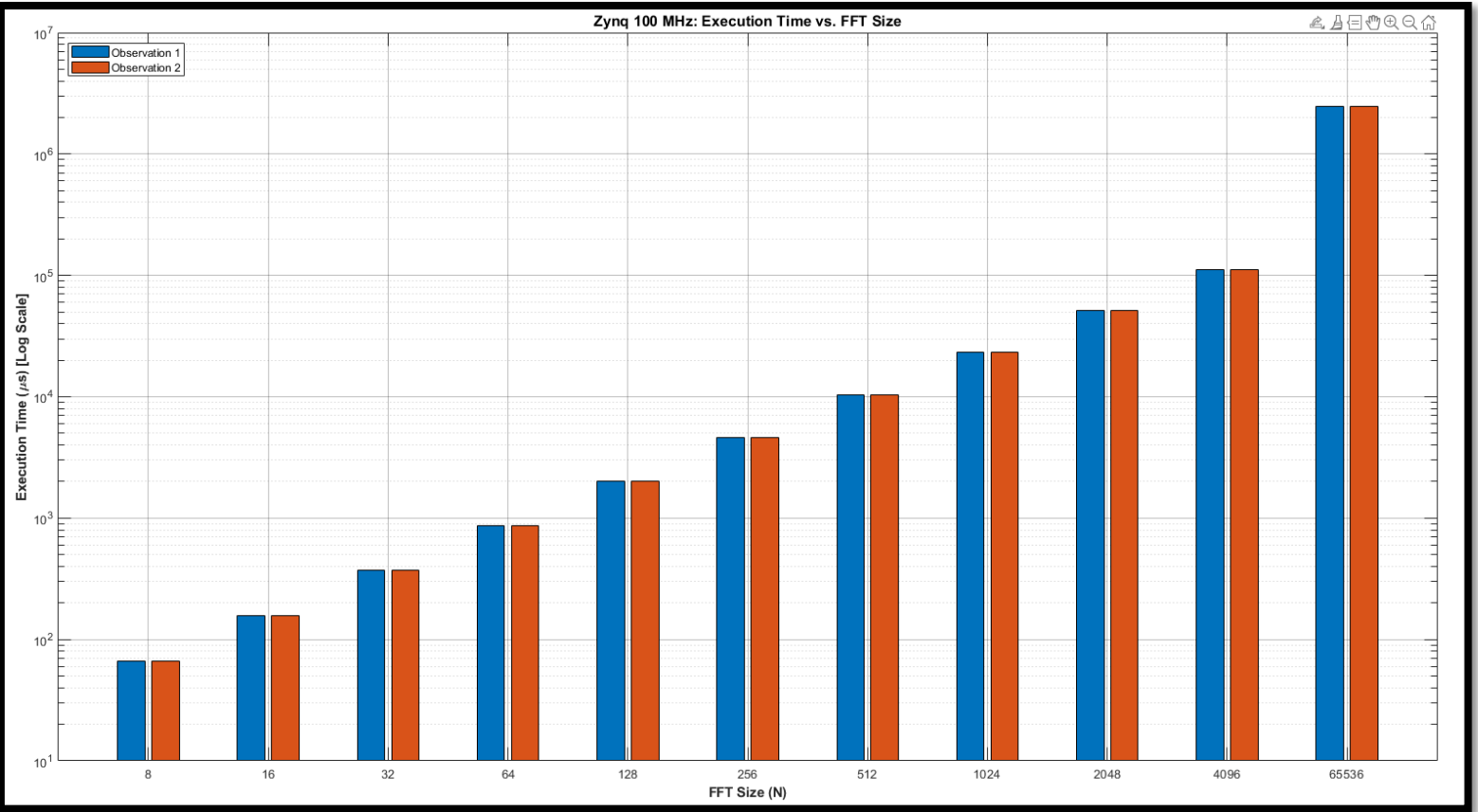
Search: Q-

Component	Clock Source	Requested Freq...	Actual Frequency(...)	Range(MHz)
Processor/Memory Clocks				
CPU	ARM PLL	100.0000000	100.000000	50.0 : 800.0
DDR	DDR PLL	533.333313	533.333374	200.000000 : 534.000000
IO Peripheral Clocks				
PL Fabric Clocks				
System Debug Clocks				
Timers				

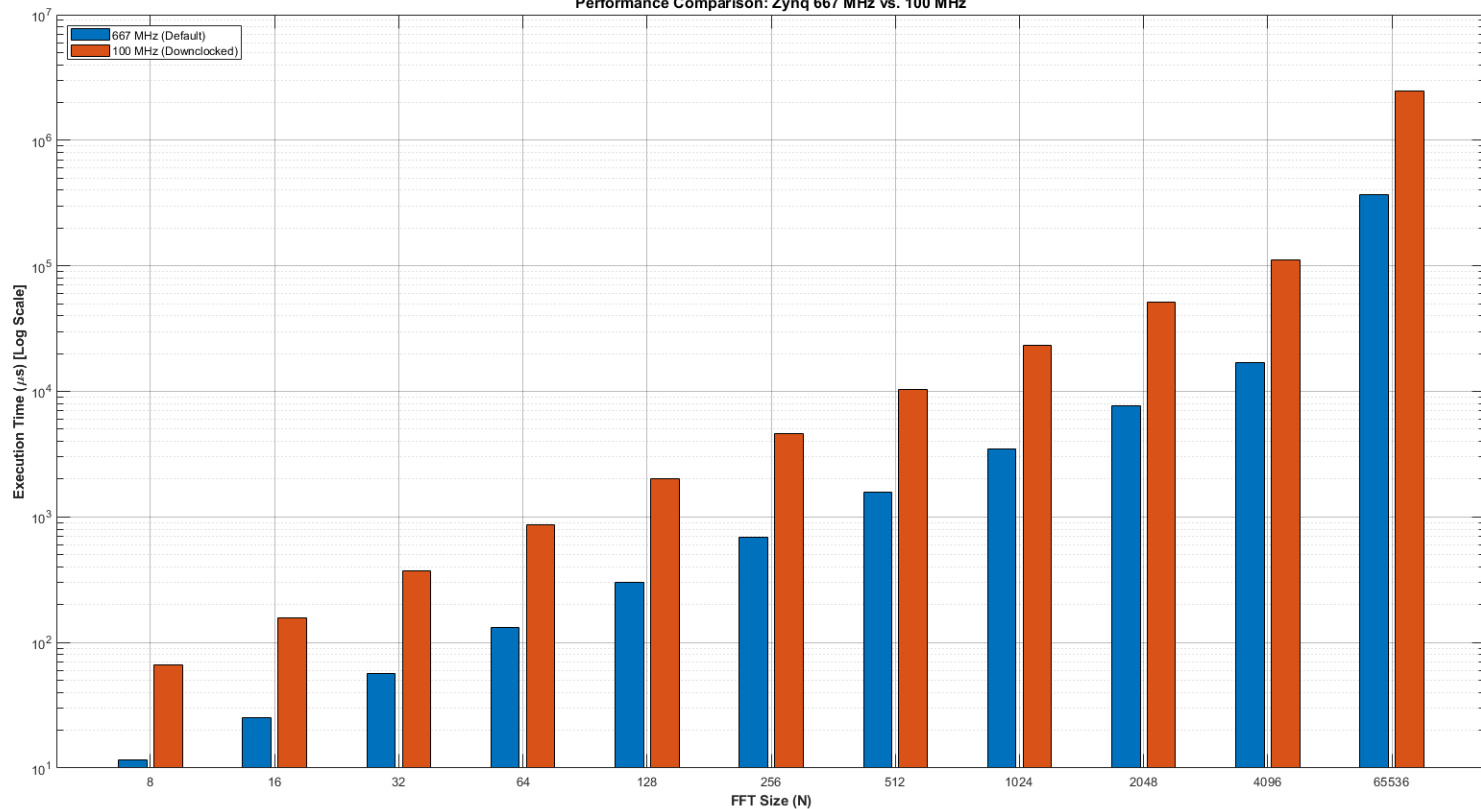
(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= N	int	65536
> FFT_input	complex float *	0x00103cd8
> FFT_output	complex float *	0x00183ce0
(x)= start_time	XTime	0x0000000000000006
(x)= end_time	XTime	0x0000000007582439
> ptr1	complex float *	0x00183cd8
> ptr2	complex float *	0x00203ce0
> ip	complex float [65536]	
> op	complex float [65536]	
(x)= execution_time_sec	float	2.464339
(x)= execution_time_us	float	2464339

Zynq: - Clock frequency 100 MHz

FFT Size	Time (in us) (in micro seconds)	
	Observation 1	Observation 2
8	66.06300	65.88000
16	156.18000	155.94000
32	370.88000	370.12000
64	866.50000	867.78000
128	2 013.01000	2 013.68000
256	4 614.60000	4 615.62000
512	10 398.28000	10 412.64000
1024	23 269.64000	23 271.50000
2048	51 323.96000	51 209.20000
4096	112 074.30000	112 295.70000
65536	2 463 640.00000	2 463 204.00000



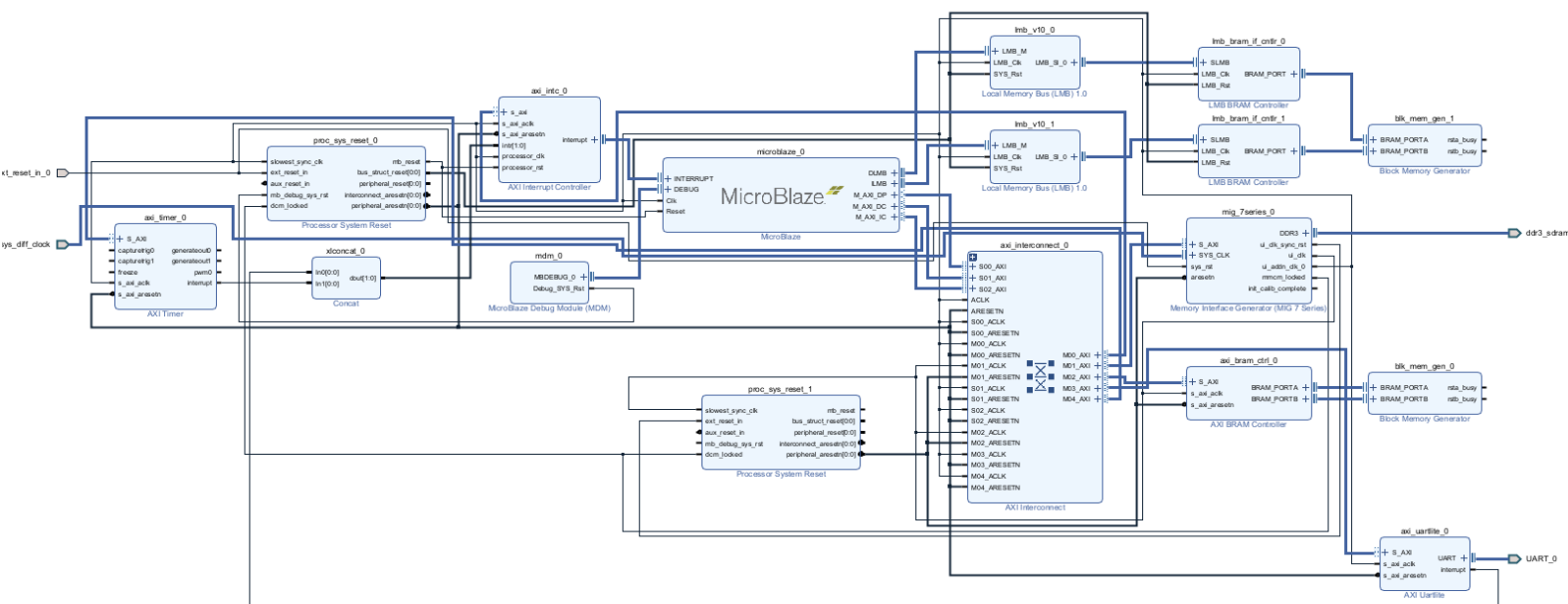
Performance Comparison: Zynq 667 MHz vs. 100 MHz



MicroBlaze Softcore Processor Configurations

Because the MicroBlaze is built out of programmable logic (PL), we created several different hardware configurations, adding more advanced features each time to see how much we could speed up **(Optimization)** the calculation of FFT.

Block diagram



Double click on the IP to customize it!

AXI TIMER configuration.

Re-customize IP

AXI Timer (2.0)

Documentation IP Location

☐ Show disabled ports

Component Name

☐ Enable 64-bit mode

Width of the timer/counter (bits)

Timer 1

Active state of Capture Trigger

Active state of Generate Out signal

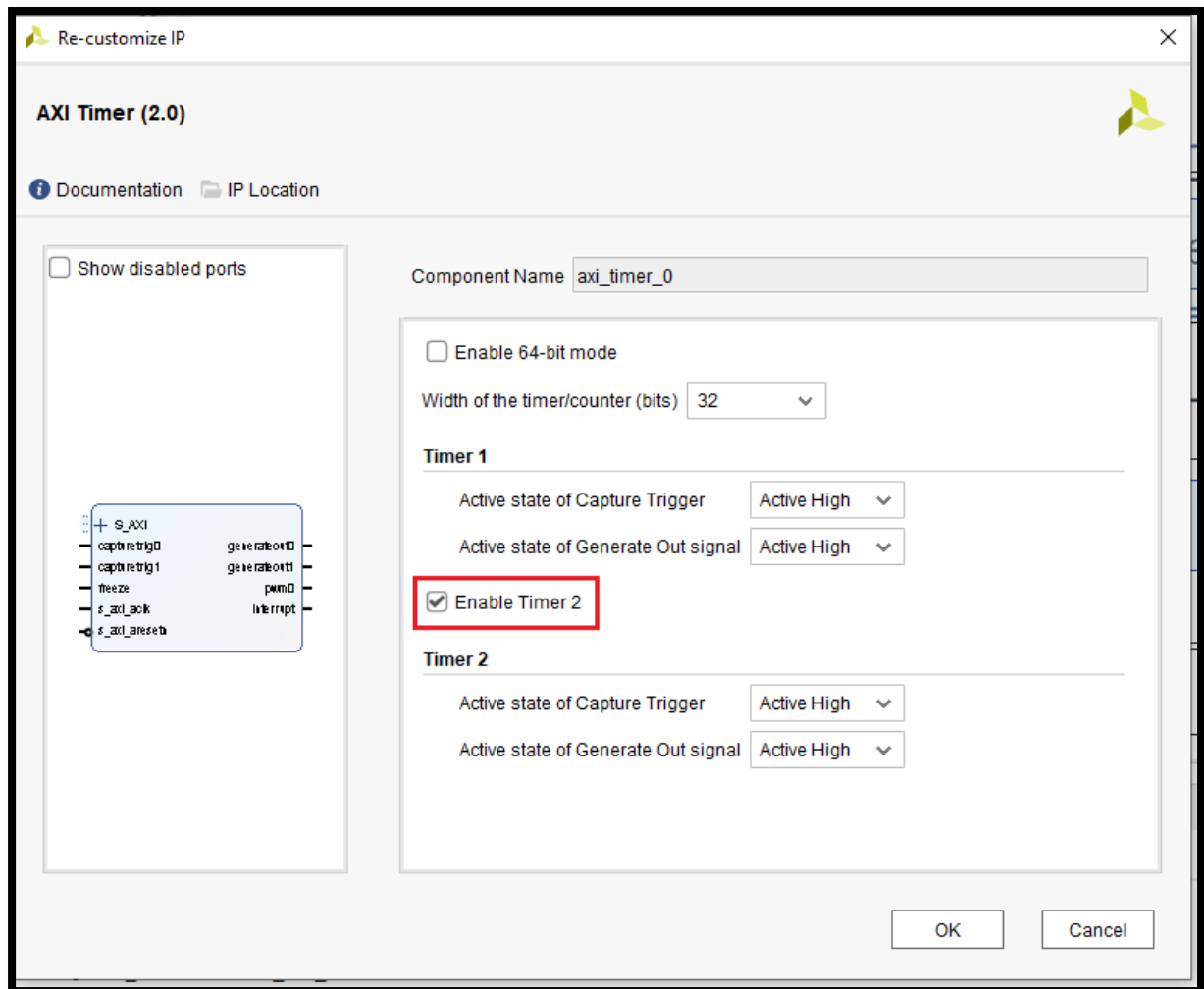
☒ Enable Timer 2

Timer 2

Active state of Capture Trigger

Active state of Generate Out signal

OK Cancel



The image shows the 'AXI Timer (2.0)' configuration window. On the left, a port connection diagram shows inputs: s_axi, capturetrig0, capturetrig1, freeze, s_axi_ack, and s_axi_resetn. On the right, the 'Component Name' is 'axi_timer_0'. The 'Enable 64-bit mode' checkbox is unchecked, and the 'Width of the timer/counter (bits)' is set to 32. Under 'Timer 1', the 'Active state of Capture Trigger' and 'Active state of Generate Out signal' are both set to 'Active High'. The 'Enable Timer 2' checkbox is checked and highlighted with a red box. Under 'Timer 2', the 'Active state of Capture Trigger' and 'Active state of Generate Out signal' are both set to 'Active High'. At the bottom right are 'OK' and 'Cancel' buttons.

Re-customize IP

MicroBlaze (11.0)

Documentation IP Location Advanced

IP Symbol Resource Estimates Component Name microblaze_0

Frequency
Area
Performance

Resource Estimates

Category	Percent (%)
Frequency	100.0
Area	~15.0
Performance	~10.0

Resource Usage

BRAM: 0 DSP48E: 0

MicroBlaze

Usage Information

- Select a predefined configuration with *Select Configuration* below. Information about the selected configuration can be found in the tooltip. *Each predefined configuration completely changes the MicroBlaze parameters.*
- To modify the configuration, click on the *Next* button, click on the *Advanced* button at the top to directly access parameters in a tabbed interface, or click *OK* to accept the configuration and close the dialog.

Predefined Configurations

Select Configuration: Current Settings

Select Processor Implementation: 32 64

General Settings

Select implementation: Linux with MMU

☒ Enable MicroBlaze

☐ Use Instruction Cache

☐ Enable Exceptions

☐ Use Memory Management

☐ Enable Discrete Ports

< Back Next > Page 1 of 4

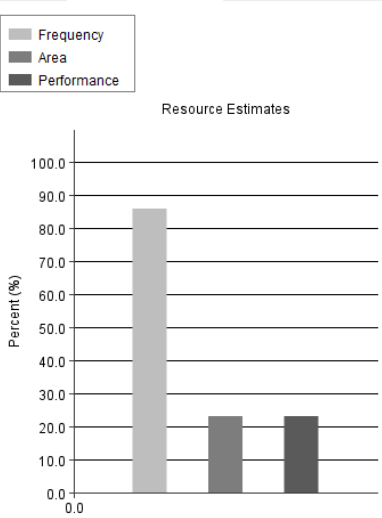
Application Preset: Instead of manually checking dozens of individual boxes to build the processor, Vivado offers "presets". The **Application Preset** automatically configures the MicroBlaze as a high-end computer(Application processor) . It instantly turns on the features required to run heavy software or operating systems, ensuring the processor is built for complex tasks right out of the box.

MicroBlaze (11.0)

Documentation IP Location Advanced

IP Symbol Resource Estimates

Component Name: microblaze_0



Resource Estimates

Percent (%)

Resource Usage

BRAM: 6 DSP48E: 0

MicroBlaze

Usage Information

- Select a predefined configuration with *Select Configuration* below. Information about the selected configuration can be found in the tooltip. *Each predefined configuration completely changes the MicroBlaze parameters.*
- To modify the configuration, click on the *Next* button, click on the *Advanced* button at the top to directly access parameters in a tabbed interface, or click *OK* to accept the configuration and close the dialog.

Predefined Configurations

Select Configuration: Current Settings

Select Processor Implementation

32 64

General Settings

Select implementation optimization: PERFORMANCE

☒ Enable MicroBlaze Debug Module Interface
☒ Use Instruction and Data Caches
☐ Enable Exceptions
☐ Use Memory Management
☐ Enable Discrete Ports

Select MicroBlaze optimization:

- When set to **PERFORMANCE**, the implementation is selected to optimize computational performance, using a five-stage pipeline.

This setting is normally recommended, unless optimizing area or system frequency is critical.

- When set to **AREA**, the implementation is selected to optimize area, using a three-stage pipeline with lower instruction throughput.

It is recommended to enable area optimization on architectures with limited resources, such as Artix.

Note that area optimization is not available together with Memory Management Unit (MMU), Branch Target Cache, Instruction Cache Streams and Victims, Data Cache Victims, or AXI Coherency Extension (ACE).

- When set to **FREQUENCY**, the implementation is selected to optimize MicroBlaze frequency, using an eight-stage pipeline.

Page 1 of 5

Cancel

Enable Instruction Cache: The processor calculates math much faster than the external DDR RAM can send it information. If the processor has to ask the DDR RAM for every single line of your C code, it spends most of its time waiting. The **Instruction Cache** is a small, super-fast memory built directly into the processor. It grabs large chunks of your C code ahead of time, so the processor always knows what to do next without waiting.

Enable Data Cache: Similar to the Instruction Cache, the **Data Cache** is a super-fast internal memory used specifically for variables and numbers. When your C code is crunching the FFT array, it brings those numbers into the Data Cache. The processor can manipulate them instantly without having to constantly send data back and forth to the slow DDR RAM.

Optimization (Performance): By setting the optimization to "Performance," we command Vivado to build a 5-stage pipeline. This means the processor acts like an assembly line; it can work on 5 different instructions at the exact same time. This takes up more physical logic gates on the FPGA, but it makes the software run significantly faster than the smaller "Area" optimization option.

FPU Basic: This gives the processor the physical circuits to do simple math with decimal numbers. It can only do Addition (+), Subtraction (-), Multiplication (*), Division (/), and comparing numbers to see which is larger.

FPU Extended: This includes everything from the Basic version, but adds extra physical circuits for advanced math. It allows the processor to instantly calculate **Square Roots** and convert whole numbers into decimal numbers right in the hardware.

Because the FFT algorithm uses very complex decimal math, choosing "Extended" gives us the maximum possible speed.

C Code of the FFT algorithm

Everything of the code is same as that of Zynq,

Only the timer code is different !

To get the exact time, we used the **AXI Timer** like a digital stopwatch.

1. **Initialize the Timer:** XTmrCtr_Initialize and XTmrCtr_SetOptions set the timer to "Cascade Mode." This joins two 32-bit timers together to create one massive 64-bit stopwatch that will never run out of space.
2. **Start the Stopwatch:** XTmrCtr_Start(&timer_inst, 0) is called exactly one line before the fft_N() function begins.
3. **Stop the Stopwatch:** XTmrCtr_Stop(&timer_inst, 0) is called immediately after the FFT math finishes.

4. **Calculate Ticks:** Since it is a 64-bit timer, we grab the bottom 32 bits and the top 32 bits, then join them together

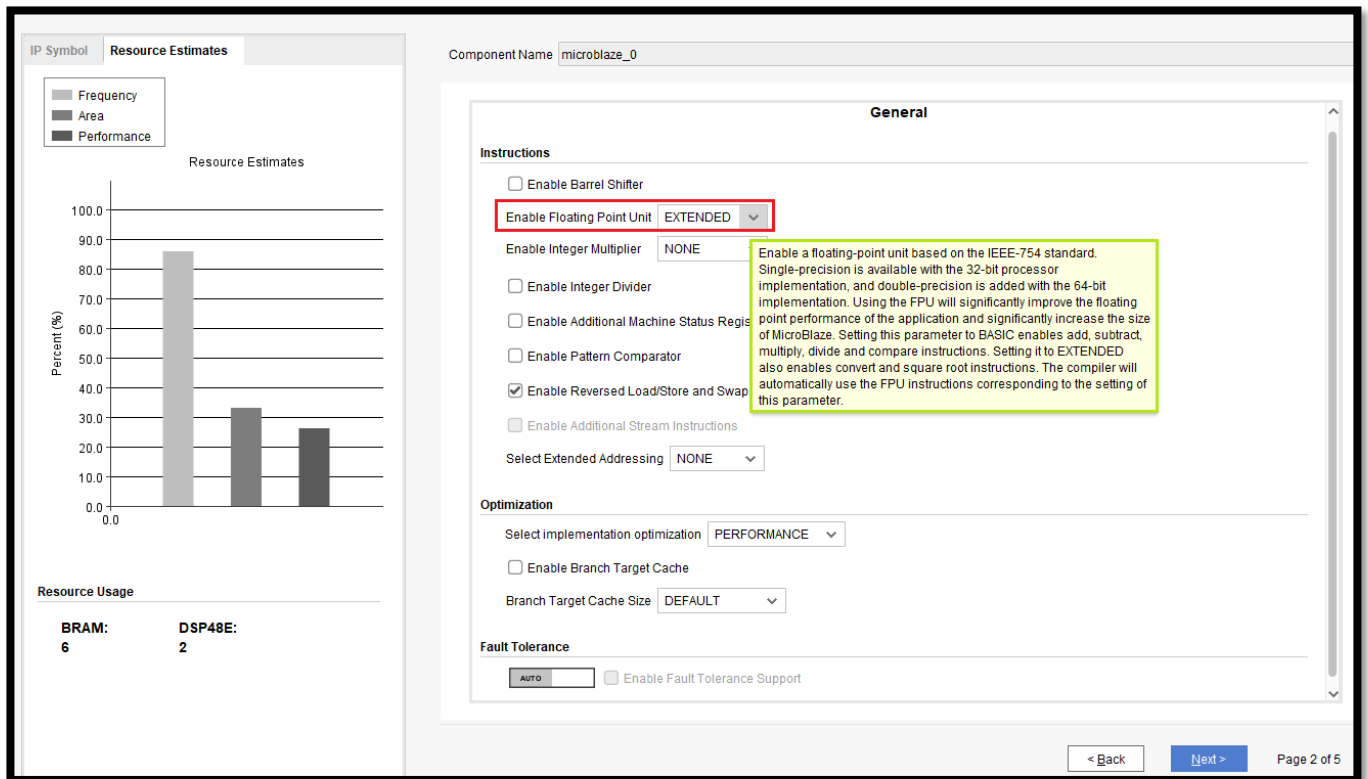
(total_ticks = (upper << 32) | lower) to get the total count.

5. **Convert to Seconds:** We divide the total_ticks by the board's clock frequency (XPAR_AXI_TIMER_0_CLOCK_FREQ_HZ) to get the actual time in seconds.

```
202 int main(){
203     int N=FFT_Size;
204     float complex* FFT_input = (float complex*) malloc (N* sizeof(float complex));
205     float complex* FFT_output= (float complex*) malloc (N* sizeof(float complex));
206
207     XTmrCtr timer_inst ; // make timer instance
208
209     XTmrCtr_Initialize(&timer_inst, XPAR_TMRCTR_0_DEVICE_ID) ; //initialize the timer
210     XTmrCtr_SetOptions(&timer_inst, XPAR_TMRCTR_0_DEVICE_ID, XTC_CASCADE_MODE_OPTION);
211     //set the timer in CASCADE mode
212     float complex* ptr1 = FFT_input ;
213     float complex* ptr2 = FFT_output ;
214
215     generate_input(FFT_input, input_sel, N ) ;
216
217     XTmrCtr_Reset(&timer_inst, 0); //reset to 0
218     XTmrCtr_Reset(&timer_inst, 1);
219
220     u32 lower_4bytes_timer = 0 , upper_4bytes_timer=0 ; //declare lower and higher 32 bits
221     u64 total_ticks =0 ; // declare the 64 bits variable
222
223     XTmrCtr_Start(&timer_inst, 0); // start TIMER
224
225     fft_N(FFT_output , N , FFT_input) ; // call the FFT function
226
227     XTmrCtr_Stop(&timer_inst, 0); //STOP TIMER
228
229     lower_4bytes_timer = XTmrCtr_GetValue(&timer_inst,0) ; //get the lower 32 bits
230     upper_4bytes_timer = XTmrCtr_GetValue(&timer_inst, 1) ; //get the upper 32 bits
231
232     total_ticks = ( (u64)upper_4bytes_timer << 32) | lower_4bytes_timer; //concatenate the lower and higher 32 bits
233     //and store it in the 64 bit variable.
234
235     float complex ip[FFT_Size] ; //declare 2 array for ip and output storage
236     float complex op[FFT_Size] ;
237
238     for (int i = 0 ; i < N ; i++){
239         ip[i] = *ptr1 ; //storing the input in ip array
240         ptr1++;
241     }
242
```

```
243     for (int i = 0 ; i < N ; i++){
244         float real2 = crealf(*ptr2)*crealf(*ptr2) ;
245         float imag2 = cimagf(*ptr2)*cimagf(*ptr2) ;
246         float magnitude = sqrt(real2+imag2) ; // calculate the magnitude of the output
247         op[i] = magnitude ; // and store in the op array
248         ptr2 ++ ;
249     }
250
251     float execution_time_sec = 0.0 ;
252     float execution_time_us = 0.0 ;
253     execution_time_sec = (float)1.0 * total_ticks / XPAR_AXI_TIMER_0_CLOCK_FREQ_HZ ;
254     execution_time_us = execution_time_sec *(1000000) ;
255
256     return 0 ;
257 }
```

Configuration A: MicroBlaze 100 MHz + FPU extended



MicroBlaze 100 MHz + FPU extended

FFT Size	Time (in sec) (in seconds)	
	Observation 1	Observation 2
8	0.01152599	0.01152749
16	0.03652686	0.03650934
32	0.10178880	0.10174160
64	0.26279800	0.26280060
128	0.64611960	0.64642460
256	1.53564500	1.53637000
512	3.55899100	3.56062700
1024	8.10066900	8.09686700
2048	18.16722000	18.15866000
4096	40.25024000	40.23150000
65536	895.21500429	894.79808984

Configuration B: MicroBlaze 100 MHz + FPU extended + Barrel Shifter

MicroBlaze (11.0)

Documentation IP Location Advanced

IP Symbol Resource Estimates

Frequency
Area
Performance

Resource Estimates

Resource	Percent (%)
Frequency	85.0
Area	35.0
Performance	35.0

Resource Usage

Resource	Usage
BRAM:	6
DSP48E:	2

Component Name: microblaze_0

General

Instructions

- ☒ Enable Barrel Shifter
- ☒ Enable Floating Point Unit EXTENDED
- ☐ Enable Integer Multiplier NONE
- ☐ Enable Integer Divider
- ☐ Enable Additional Machine Status Register Instructions
- ☐ Enable Pattern Comparator
- ☒ Enable Reversed Load/Store and Swap Instructions
- ☐ Enable Additional Stream Instructions
- Select Extended Addressing NONE

Optimization

- Select implementation optimization PERFORMANCE
- ☐ Enable Branch Target Cache
- Branch Target Cache Size DEFAULT

Fault Tolerance

- ☐ Enable Fault Tolerance Support

< Back Next > Page 2 of 5

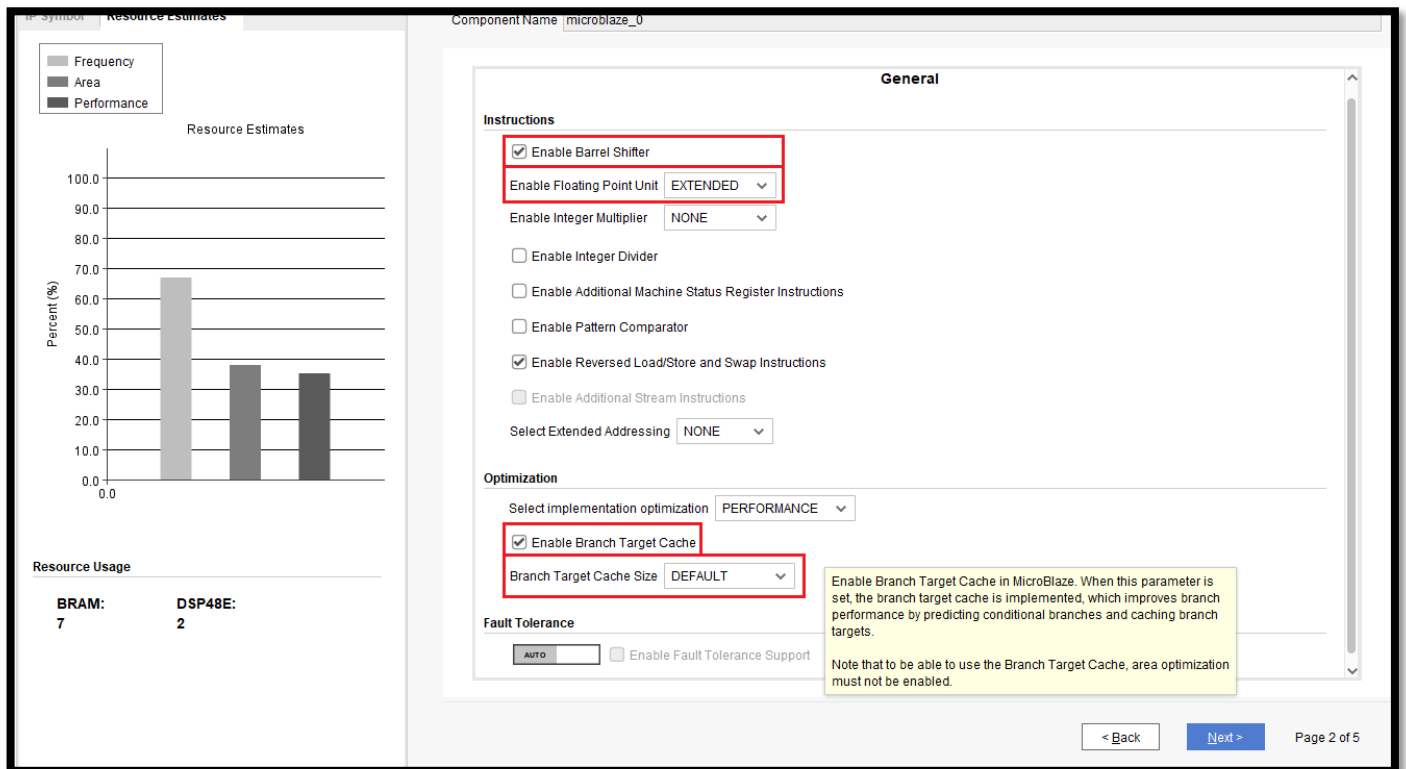
Barrel Shifter: In binary math, shifting numbers to the left or right is a fast way to multiply or divide. A barrel shifter is a special physical circuit that can shift data by any number of positions in a single clock cycle. This saves the processor a massive amount of time during repetitive math loops.

MicroBlaze 100 MHz + FPU extended + Barrel Shifter

FFT Size	Time (in sec) (in seconds)	
	<u>Observation 1</u>	<u>Observation 2</u>
8	0.00883896	0.00883840
16	0.02794850	0.02794856
32	0.07774166	0.07741650
64	0.20065650	0.20077180
128	0.49329900	0.49300580
256	1.17166900	1.17096900
512	2.71288800	2.71451000
1024	6.16954600	6.17321100
2048	13.83795000	13.82971500
4096	30.65641000	30.63813000
65536	681.42770000	681.42773000

Configuration C: Adding the Branch Target Cache (BTC)

MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of DEFAULT entries.



Branch Target Cache (BTC): When the C code runs a for loop or an if/else statement, the processor has to stop and figure out which path to take. The BTC is a predictive memory. It remembers the paths the code took previously and guesses which way it will go next.

Entries selected (Default, 512, 2048): This is the size of the prediction memory. A cache with 2048 entries can remember more paths than a cache with 512 entries, making the processor's guesses more accurate.

MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of DEFAULT entries

FFT Size	Time (in sec) (in seconds)	
	<u>Observation 1</u>	<u>Observation 2</u>
8	0.00865831	0.00865949
16	0.02735662	0.02736223
32	0.07610202	0.07605878
64	0.19622090	0.19622380
128	0.48199090	0.48225350
256	1.14446100	1.14500700
512	2.65090700	2.65091500
1024	6.03102000	6.02777260
2048	13.50949000	13.51723500
4096	29.94144000	29.92502000
65536	665.35360000	665.35830000

Configuration D: MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of 512 entries.

Frequency

Area

Performance

Resource Estimates

Resource Usage

BRAM: 7

DSP48E: 2

General

Instructions

☒ Enable Barrel Shifter

Enable Floating Point Unit EXTENDED

Enable Integer Multiplier NONE

☐ Enable Integer Divider

☐ Enable Additional Machine Status Register Instructions

☐ Enable Pattern Comparator

☒ Enable Reversed Load/Store and Swap Instructions

☐ Enable Additional Stream Instructions

Select Extended Addressing NONE

Optimization

Select Implementation optimization PERFORMANCE

☒ Enable Branch Target Cache

Branch Target Cache Size 512 ENTRIES

Fault Tolerance

AUTO

Enable

< Back

Next >

Page 2 of

OK

Cancel

(x)= Variables Breakpoints Expressions Registers		
Name	Type	Value
(x)= N	int	512
> FFT_input	complex float *	0x80000008
> FFT_output	complex float *	0x80001010
> timer_inst	XTmrCtr	{Config={DeviceId=0x0000, BaseAddress=0x41c00000,
> ptr1	complex float *	0x80001008
> ptr2	complex float *	0x80002010
(x)= lower_4bytes_timer	u32	0x0f66ec6b
(x)= upper_4bytes_timer	u32	0x00000000
(x)= total_ticks	u64	0x000000000f66ec6b
> ip	complex float [512]	[1.000000, 0.9980267+0.06279052i, 0.9921147+0.1253
> op	complex float [512]	[11.71970, 14.56346, 19.23035, 28.30034, 53.56742,
(x)= execution_time_sec	float	2.584034
(x)= execution_time_us	float	2584034

MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of 512 entries

FFT Size	Time (in sec) (in seconds)	
	<u>Observation 1</u>	<u>Observation 2</u>
8	0.00831557	0.00831969
16	0.02643448	0.02643546
32	0.07375826	0.07375921
64	0.19066260	0.19066060
128	0.46902300	0.46912100
256	1.11473800	1.11463200
512	2.58546800	2.58547200
1024	5.87898900	5.87897200
2048	13.1811300	13.1812500
4096	29.2074500	29.2074800
65536	649.5779000	649.5776000

Configuration E: MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of 2048 entries.

Resource Estimates

Percent (%)

100.0
90.0
80.0
70.0
60.0
50.0
40.0
30.0
20.0
10.0
0.0

Area
Performance

Resource Usage

BRAM: 9
DSP48E: 2

General

Instructions

- ☒ Enable Barrel Shifter
- Enable Floating Point Unit: EXTENDED
- Enable Integer Multiplier: NONE
- ☐ Enable Integer Divider
- ☐ Enable Additional Machine Status Register Instructions
- ☐ Enable Pattern Comparator
- ☒ Enable Reversed Load/Store and Swap Instructions
- ☐ Enable Additional Stream Instructions
- Select Extended Addressing: NONE

Optimization

Select implementation optimization: PERFORMANCE

- ☒ Enable Branch Target Cache
- Branch Target Cache Size: 2048 ENTRIES

Fault Tolerance

AUTO ☐ Enable

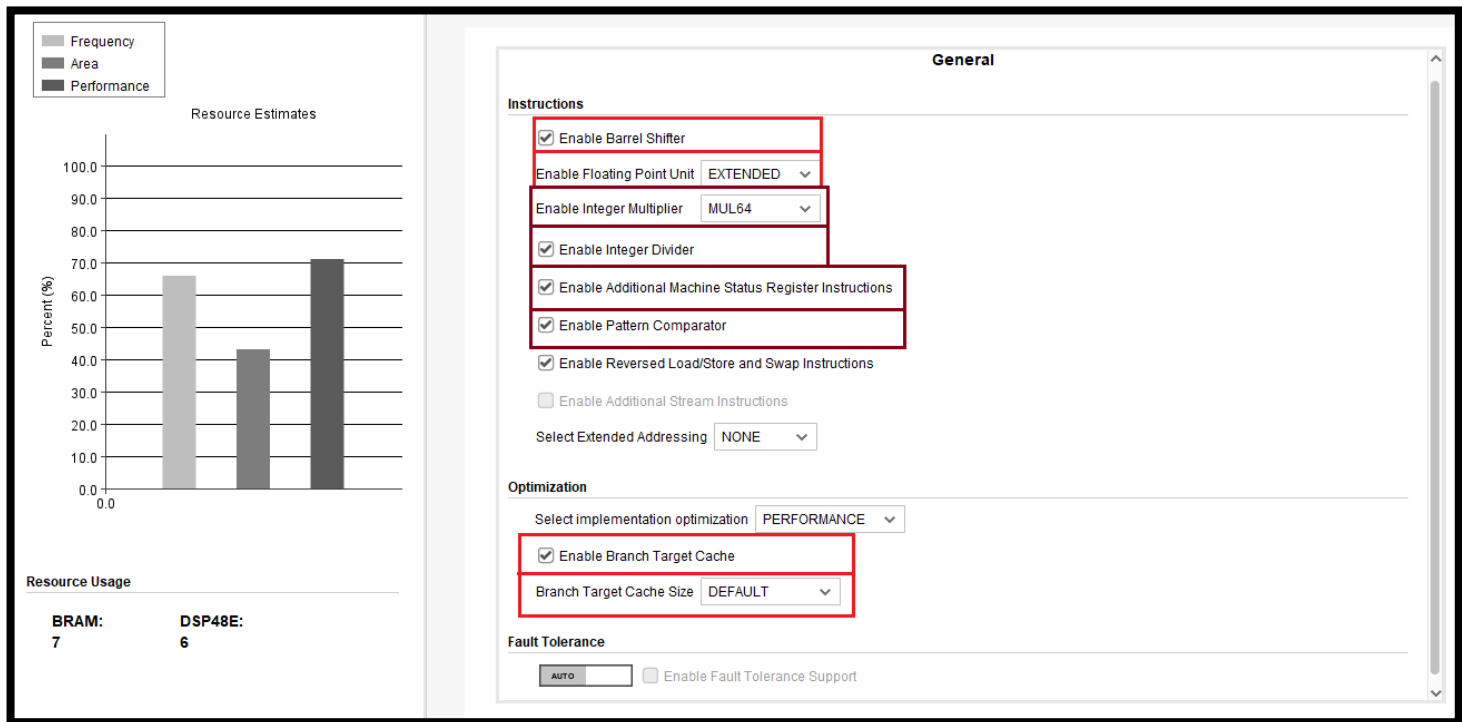
< Back Next > Page 2 of 5

OK Cancel

MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of 2048 entries

FFT Size	Time (in sec) (in seconds)	
	Observation 1	Observation 2
8	0.00831459	0.00831034
16	0.02639559	0.02640970
32	0.07368743	0.07368636
64	0.19052620	0.19042070
128	0.46865250	0.46860090
256	1.11320600	1.11384600
512	2.58041200	2.58191100
1024	5.87415300	5.87082000
2048	13.16355000	13.16858000
4096	29.16667000	29.16807000
65536	649.07060000	649.11500000

Configuration F: MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of DEFAULT entries+ Enable Integer Multiplier MUL 64 + Enable Integer Divider+ Enable additional Machine Status Register + Enable Pattern Comparator



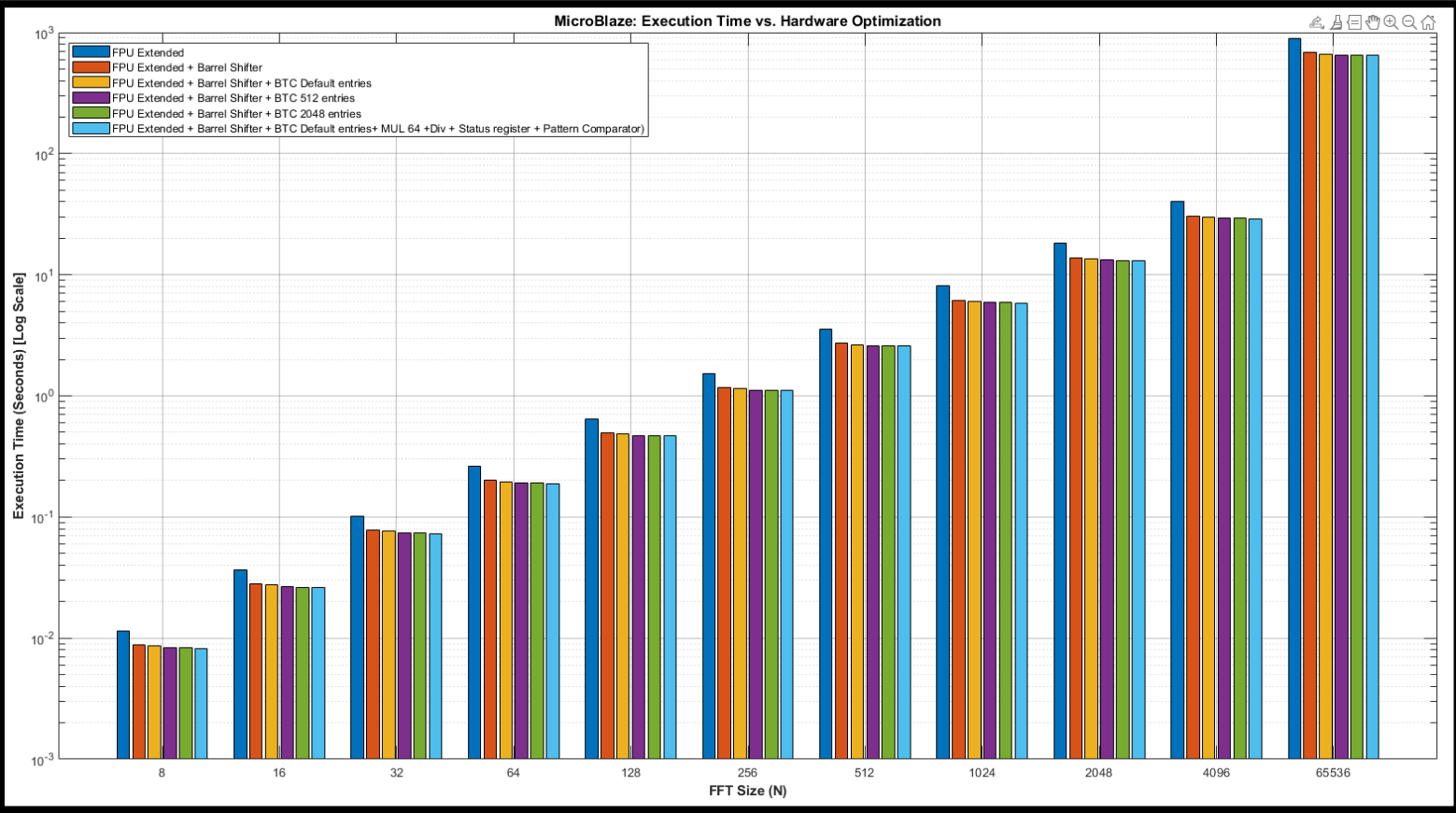
- **Integer Divider:** Normally, a processor is very bad at dividing numbers. It usually has to do a long, complex loop of subtractions to solve something like $10 / 2$. With the **Integer Divider** hardware added, the processor can solve that math problem in a single clock cycle.
- **Pattern Comparator:** This is a special circuit used to quickly scan through memory to find a specific sequence of data. If your FFT code is constantly looking for specific markers or headers in your data arrays, the **Pattern Comparator** does this search instantly in hardware rather than the software having to check every single address one by one.

- **Integer Multiplier (MUL64):** Even though the MicroBlaze is a 32-bit processor, the **MUL64** option gives it the physical logic to multiply two 32-bit numbers and get a massive 64-bit result without losing any detail. This is critical for the FFT because multiplying complex numbers creates very large results that would cause errors if they weren't stored correctly.
- **Additional Machine Status Register Instructions:** These are special "shortcuts" for the processor. They allow the C code to check the status of the CPU (like if an error occurred or if a calculation finished) using fewer steps. It makes the "administrative" work of the processor much faster, leaving more time for the actual math.

By enabling these four options, you are offloading the boring, repetitive parts of the software math directly to the FPGA's hardware, which is why your FFT execution time dropped significantly.

MicroBlaze 100 MHz + FPU extended + Barrel Shifter+ Branch Target Cache of DEFAULT entries + Enable Integer Multiplier MUL 64 + Enable Integer Divider+ Enable additional Machine Status Register + Enable Pattern Comparator.

FFT Size	Time (in sec) (in seconds)	
	<u>Observation 1</u>	<u>Observation 2</u>
8	0.00825270	0.00825560
16	0.02623858	0.02623828
32	0.07327191	0.07327201
64	0.18947520	0.18947630
128	0.46612710	0.46611210
256	1.10789000	1.10792000
512	2.56694100	2.56699100
1024	5.84013700	5.84334300
2048	13.09528000	13.10261000
4096	29.03225000	29.01632000
65536	645.74120000	645.38460000



Problems Faced and Solutions (MicroBlaze Optimization)

While upgrading the MicroBlaze processor with extra hardware drastically improved our FFT calculation speeds, we got some challenges.

1. Branch Target Cache

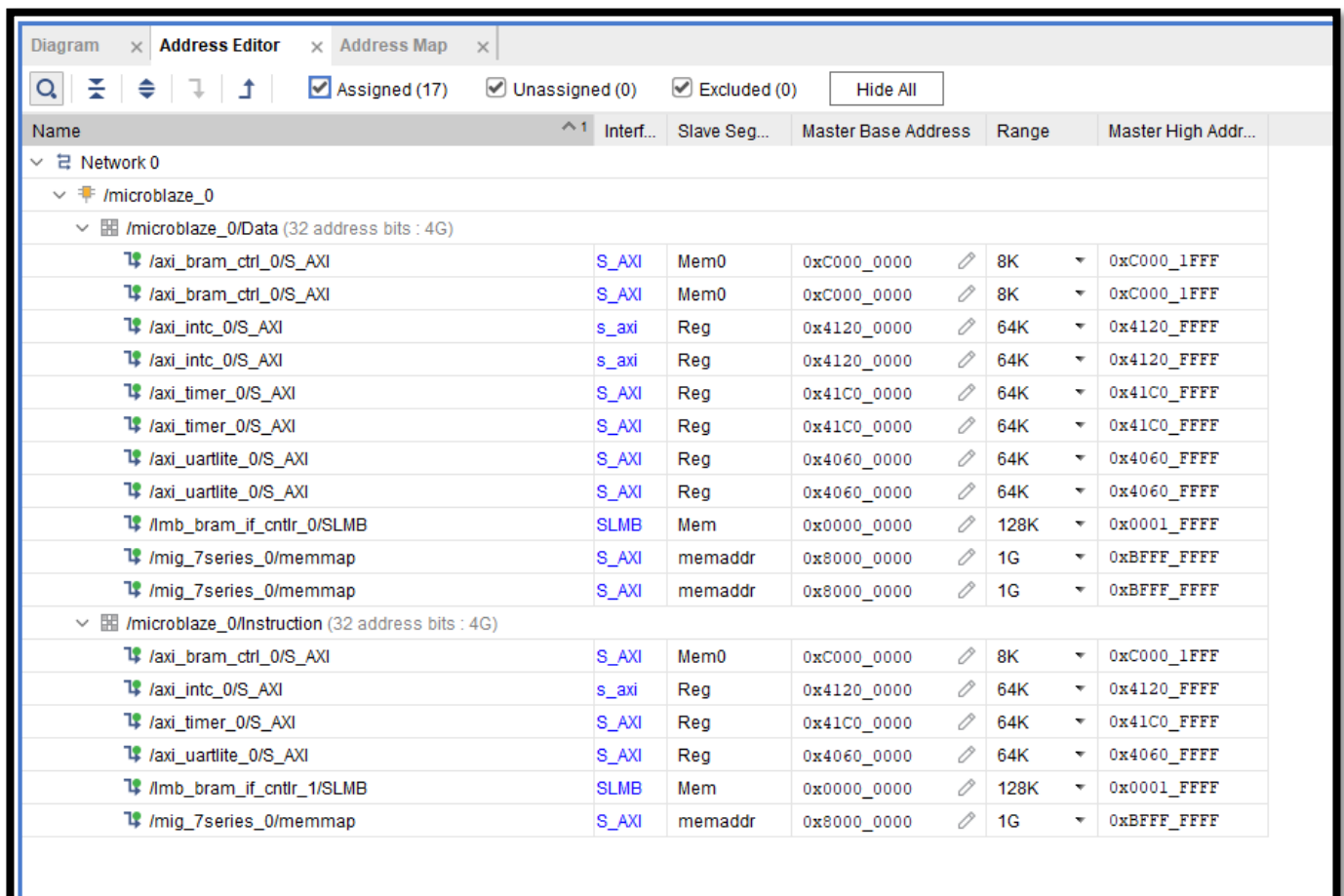
- **The Problem:** We assumed that making the Branch Target Cache (BTC) as large as possible (2048 entries) would result in a massive speed boost. However, when we looked at the data for the 65,536-point FFT, the 512-entry cache took 649.5 seconds, and the 2048-entry cache took 649.0 seconds. The speed difference was almost zero!
- **The Solution:** We realized that the FFT algorithm is highly repetitive. Once the processor learns the main math loops, it doesn't need a massive memory to remember them. We concluded that the **512-entry BTC** is the most efficient choice, as it provides maximum speed without wasting valuable FPGA memory.

2. Software Ignoring the Hardware (Compiler Flags)

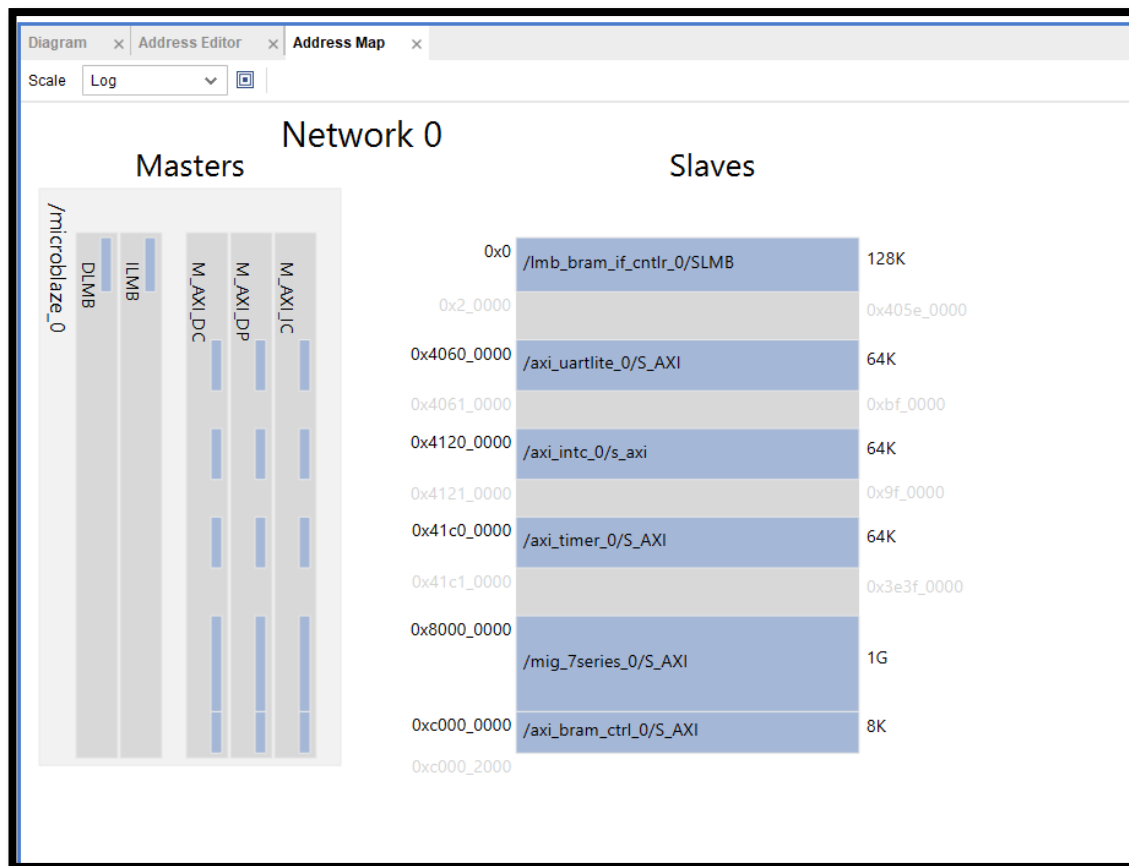
- **The Problem:** In our early tests, we added the FPU (Floating Point Unit) hardware in Vivado, but the execution time in Vitis did not get any faster. The processor was still trying to do the decimal math slowly in software.
- **The Solution:** We discovered that the Vitis C compiler did not automatically know we upgraded the hardware. We had to ensure the software project was completely rebuilt and the Board Support Package (BSP) was updated so the compiler would generate instructions to actually use the new FPU and Hardware Dividers.

3. Data Cache Overload on Huge Arrays

- **The Problem:** The execution time dropped beautifully for smaller FFT sizes. However, when we tested the massive 65,536-size FFT, the time spiked up heavily across all configurations (taking over 10 minutes even on the best MicroBlaze setup).
- **The Solution:** We identified this as a memory bottleneck. The MicroBlaze's internal Data Cache is small. A 65,536-point complex array is too massive to fit inside it. So, we increased the size of cache memory from the address editor.



Name	Interf...	Slave Seg...	Master Base Address	Range	Master High Addr...
Network 0					
/microblaze_0					
/microblaze_0/Data (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/lmb_bram_if_cntlr_0/SLMB	SLMB	Mem	0x0000_0000	128K	0x0001_FFFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF
/microblaze_0/Instruction (32 address bits : 4G)					
/axi_bram_ctrl_0/S_AXI	S_AXI	Mem0	0xC000_0000	8K	0xC000_1FFF
/axi_intc_0/S_AXI	s_axi	Reg	0x4120_0000	64K	0x4120_FFFF
/axi_timer_0/S_AXI	S_AXI	Reg	0x41C0_0000	64K	0x41C0_FFFF
/axi_uartlite_0/S_AXI	S_AXI	Reg	0x4060_0000	64K	0x4060_FFFF
/lmb_bram_if_cntlr_1/SLMB	SLMB	Mem	0x0000_0000	128K	0x0001_FFFF
/mig_7series_0/memmap	S_AXI	memaddr	0x8000_0000	1G	0xBFFF_FFFF



Generate a linker script

Generate linker script

Control your application's memory map.

Output Settings

Project: microBlaze_kapish_fft2_optimization

Output Script:

sh_vitis\workspace3\microBlaze_kapish_fft2_optimization\src\lscript.ld

Modify project build settings as follows:

Set generated script on all project build configurations

Hardware Memory Map

Memory	Base Address	Size
axi_bram_ctrl_0_Mem0	0xC0000000	8 KB
lmb_bram_if_cntlr_1_Mem_lmb_br...	0x00000000	128 KB
mig_7series_0_memaddr	0x80000000	1 GB

Basic **Advanced**

Place Code Sections in: lmb_bram_if_cntlr_1_Mem_lmb_bram_if_cntlr_0_Mem

Place Data Sections in: lmb_bram_if_cntlr_1_Mem_lmb_bram_if_cntlr_0_Mem

Place Heap and Stack in: mig_7series_0_memaddr

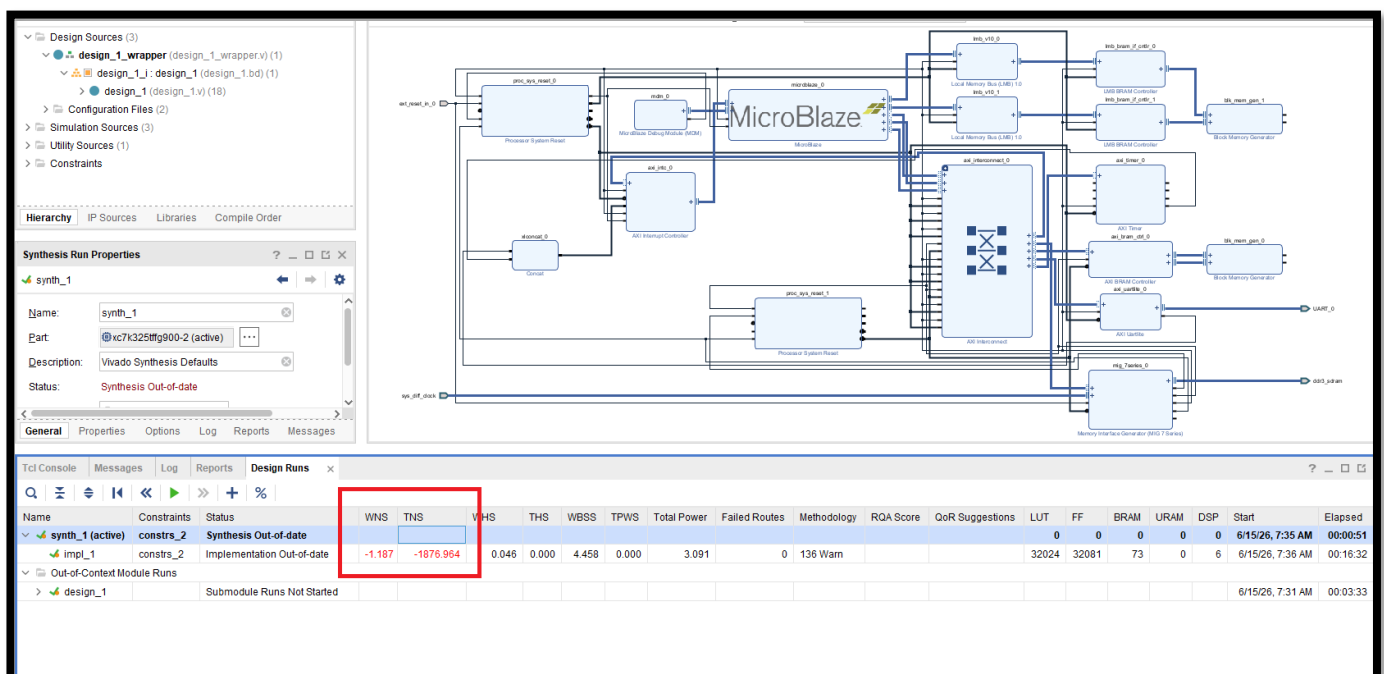
Heap Size: 10 MB 0x00A00000

Stack Size: 10 MB 0x00A00000

239 ip[i] = *ptr1 ; //storing the input ir

4. The Physical Speed Limit (Timing Violations at 200 MHz)

- **The Problem:** Because the processor performed so well at 100 MHz, we attempted to double the clock frequency to 200 MHz to see if we could cut the execution time in half again. However, Vivado failed to build the design and threw a "Timing Error" (Negative Slack).
- **The Solution:** We kept the clock speed back down to **100 MHz**, which gave the processor 10 nanoseconds per tick. This gave the electrical signals plenty of time to travel completely through the complex hardware without failing, proving that 100 MHz was the maximum stable speed for our fully upgraded MicroBlaze.



5. The "Clock Hierarchy" Problem (IP Configuration Order)

- **The Problem:** When we first tried to build the design, we added the MicroBlaze processor before the Memory Interface Generator (MIG). This caused the "Connection Automation" tool to fail or incorrectly wire the system clock. We kept getting errors about "unconnected clocks" or "timing constraints" when we tried to synthesize the design.

- **Why it Happened:** In an FPGA, the MIG is the "master" of the external DDR memory and is responsible for managing the high-speed clock tree. If you add the MicroBlaze first, Vivado doesn't know where the main memory clock is coming from. Because the MicroBlaze needs to be perfectly synchronized with the MIG to read and write to the RAM without data corruption, the clock *must* flow from the MIG to the MicroBlaze.
- **The Solution:** We adopted a "Bottom-Up" design approach. We always add and configure the **MIG IP first**. This allows Vivado to establish the physical memory interface and generate the system clock signal. Once the MIG is on the canvas, we add the **MicroBlaze IP**. Now, when we click "Run Connection Automation," Vivado can clearly see the clock source from the MIG and automatically route it to the MicroBlaze, ensuring they are perfectly synchronized .